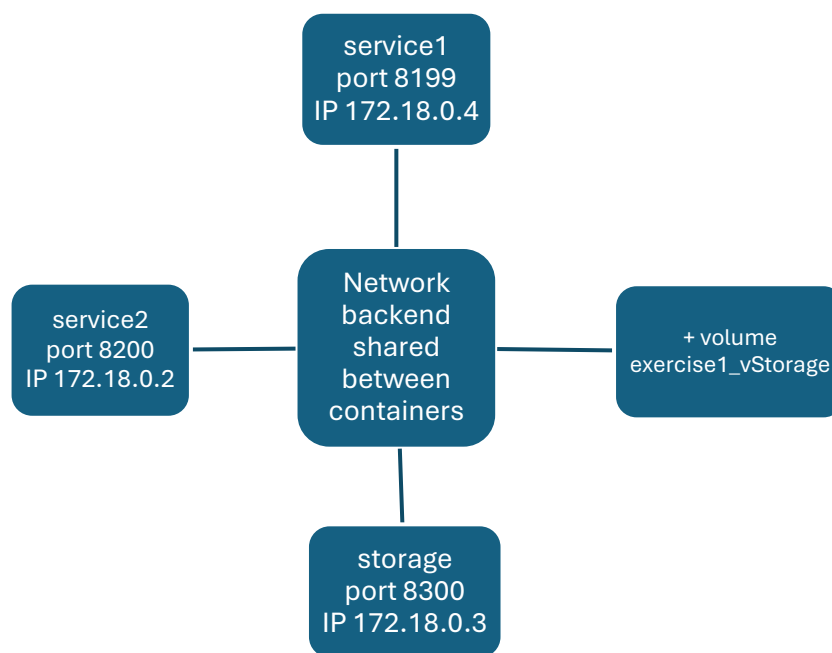


Report

Platform:

- Dell Laptop with Microsoft Windows 11 Pro
- Docker version 28.4.0
- Docker Compose version v2.39.2-desktop.1



Status record:

- Uptime in hours. Tells the container lifetime, it shows how long the container has been running. It could be useful cause the time would reset if the container crashed.
- Disk space measures the free space on the container's root filesystem. I don't think the data is that useful, it only tells how much disk space there is in general.
 - The free space could be measured for the volume and persistent storage to see the actual free space.

Storage solutions:

- Volumes:
 - Quite easy to use, since docker itself handles most of the work. When you get it created correctly and can save data, you don't need to think about it much.

- Easy way to share data in multi-container applications.
- Lifecycle is easy to handle, since it is taken care of with the whole docker compose.
- Harder to browse.
- If some issues occur, they are probably harder to solve.
- Persistent storage:
 - Transparent and easy to browse.
 - Easy for local development.
 - Need to be precise with the path.
 - The path differences between Linux vs Windows
 - The data needs to be cleared manually.

Erasing data:

- On Linux machine this should work, but I couldn't test it
 - `> ./Storage/data/log.txt`
- On windows this call can be used:
 - `Clear-Content .\Storage\data\log.txt`
- While docker container is still running, this command can be used to erase log.txt file
 - `docker compose exec storage sh -lc 'truncate -s 0 /app/data/log.txt || : > /app/data/log.txt'`

What was difficult:

- Building the Service1 took quite a lot of effort.
- Not much experience of proxys so it was something to investigate.
- Good recap indeed for the principles of TCP/IP and HTTP-protocols because I had quite well forgotten everything.

Main problems:

- There is always some problems with Dockerfiles :D
 - I tried to take some old projects as an example